An Analysis of Reinforcement learning

Reinforcement learning

Reinforcement learning -- Part 1

Safe reinforcement learning Safe reinforcement learning (SRL) can be defined as the process of learning policies that maximize the expectation of the return in problems in which it is important to ensure reasonable system performance and/or respect safety constraints during the learning and/or deployment processes. An alternative approach is risk-averse reinforcement learning, where instead of the expected return, a risk-measure of the return is optimized, such as the conditional value at risk (CVaR). In addition to mitigating risk, the CVaR objective increases robustness to model uncertainties. However, CVaR optimization in risk-averse RL requires special care, to prevent gradient bias and blindness to success.

Reinforcement learning -- Part 2

Theory Both the asymptotic and finite-sample behaviors of most algorithms are well understood. Algorithms with provably (i.e. in a way that can be proved) good online performance (addressing the exploration issue) are known. Efficient exploration of Markov decision processes is given in Burnetas and Katehakis (1997). Finite-time performance bounds have also appeared for many algorithms, but these bounds are expected to be rather loose and thus more work is needed to better understand the relative advantages and limitations. For incremental algorithms, asymptotic convergence issues have been settled. Temporal-difference-based algorithms converge under a wider set of conditions than was previously possible (for example, when used with arbitrary, smooth function approximation).

Reinforcement learning -- Part 3

Associative reinforcement learning Associative reinforcement learning tasks combine facets of stochastic learning automata tasks and supervised learning pattern classification tasks. In associative reinforcement learning tasks, the learning system interacts in a closed loop with its environment.

Reinforcement learning -- Part 4

Bias and reward function issues Designing appropriate reward functions is critical in RL because poorly designed reward functions can lead to unintended behaviors. In addition, RL systems trained on biased data may perpetuate existing biases and lead to discriminatory or unfair outcomes. Both of these issues requires careful consideration of reward structures and data sources to ensure fairness and desired behaviors.

Reinforcement learning -- Part 5

π : A × S → [0 , 1] π (a , s) = Pr (A t = a ■ S t = s) {\displaystyle {\begin{aligned}&\pi :{\mathcal {A}}\times {\mathcal {S}}\to [0,1]\\&\pi (a,s)=\Pr(A_{t}{=}a\mid S_{t}{=}s)\end{aligned}}}

Reinforcement learning -- Part 6

where G {\displaystyle G} now stands for the random discounted return associated with first taking action a {\displaystyle a} in state s {\displaystyle s} and following π {\displaystyle \pi } , thereafter. The theory of Markov decision processes states that if π ∗ {\displaystyle \pi ^{*}} is an optimal policy, we act optimally (take the optimal action) by choosing the action from Q π ∗ (s , ⋅) {\displaystyle Q^{\pi ^{*}}(s,\cdot)} with the highest action-value at each state, s {\displaystyle s} . The action-value function of such an optimal policy (Q π ∗ {\displaystyle Q^{\pi ^{*}}}) is called the optimal action-value function and is commonly denoted by Q ∗ {\displaystyle Q^{*}} . In summary, the knowledge of the optimal action-value function alone suffices to know how to act optimally. Assuming full knowledge of the Markov decision process, the two basic approaches to compute the optimal action-value function are value iteration and policy iteration. Both algorithms compute a sequence of functions Q k {\displaystyle Q_{k}} (k = 0 , 1 , 2 , … {\displaystyle k=0,1,2,\ldots }) that converge to Q ∗ {\displaystyle Q^{*}} . Computing these functions involves computing expectations over the whole state-space, which is impractical for all but the smallest (finite) Markov decision processes. In reinforcement learning methods, expectations are approximated by averaging over samples and using function approximation techniques to cope with the need to represent value functions over large state-action spaces.

Reinforcement learning -- Part 7

The first problem is corrected by allowing the procedure to change the policy (at some or all states) before the values settle. This too may be problematic as it might prevent convergence. Most current algorithms do this, giving rise to the class of generalized policy iteration algorithms. Many actor-critic methods belong to this category. The second issue can be corrected by allowing trajectories to contribute to any state-action pair in them. This may also help to some extent with the third problem, although a better solution when returns have high variance is Sutton's temporal difference (TD) methods that are based on the recursive Bellman equation. The computation in TD methods can be incremental (when after each transition the memory is changed and the transition is thrown away), or batch (when the transitions are batched and the estimates are computed once based on the batch). Batch methods, such as the

least-squares temporal difference method, may use the information in the samples better, while incremental methods are the only choice when batch methods are infeasible due to their high computational or memory complexity. Some methods try to combine the two approaches. Methods based on temporal differences also overcome the fourth issue. Another problem specific to TD comes from their reliance on the recursive Bellman equation. Most TD methods have a so-called λ {\displaystyle \lambda } parameter (0 ≤ λ ≤ 1) {\displaystyle (0\leq \lambda \leq 1)} that can continuously interpolate between Monte Carlo methods that do not rely on the Bellman equations and the basic TD methods that rely entirely on the Bellman equations. This can be effective in palliating this issue.

Reinforcement learning -- Part 8

Sample inefficiency RL algorithms often require a large number of interactions with the environment to learn effective policies, leading to high computational costs and time-intensive to train the agent. For instance, OpenAI's Dota-playing bot utilized thousands of years of simulated gameplay to achieve human-level performance. Techniques like experience replay and curriculum learning have been proposed to deprive sample inefficiency, but these techniques add more complexity and are not always sufficient for real-world applications.

Reinforcement learning -- Part 9

In machine learning and optimal control, reinforcement learning (RL) is concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. Reinforcement learning is one of the three basic machine learning paradigms, alongside supervised learning and unsupervised learning. While supervised learning and unsupervised learning algorithms respectively attempt to discover patterns in labeled and unlabeled data, reinforcement learning involves training an agent through interactions with its environment. To learn to maximize rewards from these interactions, the agent makes decisions between trying new actions to learn more about the environment (exploration), or using current knowledge of the environment to take the best action (exploitation). The search for the optimal balance between these two strategies is known as the exploration–exploitation dilemma.